ENGENHARIA DE CONFIABILIDADE · GUIA PRÁTICO

K6: do primeiro script ao teste de performance

Como transformar carga em rotina de engenharia com código.

Thauã, Gean, Allana e Talvan



Made with GAMMA

ANTES DO K6

Antes do K6: que problema o teste de performance resolve?

Teste de desempenho observa como a aplicação se comporta quando recebe carga de trabalho: requisições e usuários simultâneos.

Problema

Sem teste, gargalos aparecem só quando a demanda cresce: respostas lentas, erros crescentes, filas e conexões acumuladas.

Medição

Observar latência, throughput, taxa de erros e consumo de recursos sob carga.

Decisão

Trocar "o sistema aguenta?" por evidência observável e critérios de aprovação.

Made with GAMMA

CONTEXTO E ORIGEM

Em 2016, existia um gap entre o simples demais e o complexo demais

1. **Ferramentas simples, mas limitadas** · ApacheBench e similares resolviam cenários básicos, mas ofereciam pouca flexibilidade para testes de carga mais completos.
2. **Ferramentas completas, mas pesadas** · JMeter e LoadRunner eram aplicativos GUI antigos, grandes e desajeitados, com esforço significativo para aprender e operar.
3. **O gap** · Desenvolvedores acabavam fazendo poucos testes de carga, porque as ferramentas não compensavam o esforço de adoção.



Em 2016, a startup Load Impact criou o K6 para tornar testes de carga mais acessíveis, scriptáveis e próximos do trabalho de desenvolvimento.

Made with GAMMA

REQUISITOS

Do gap aos requisitos: o que o K6 precisava ser

1 Scriptável em linguagem popular

Testes escritos em **código real**, sem aprender uma DSL limitada.

2 Alto desempenho

Executa testes em hardware limitado e continua útil. Ferramenta **leve** por design.

3 Idioma popular

Escrito em um idioma popular, para facilitar contribuições da **comunidade**.

4 Adequado para automação

API de scripts amigável, resultados de aprovação ou falha e saída padronizada, analisável por computador. Pronto para **automação**.

5 Projeto em código aberto

Permite evolução colaborativa, ajuda da comunidade e mais interação com a ferramenta. Um projeto **aberto**.

Made with GAMMA

HISTÓRIA DO K6

A trajetória do K6: de ferramenta de carga a plataforma de observabilidade

2016

Surge com a LoadImpact

Criado para tornar testes de carga mais acessíveis.

2017

Código aberto

Lançamento como projeto de código aberto no GitHub.

2020

Rebranding

A LoadImpact muda seu nome para K6.

2021

Grafana Labs

A Grafana Labs adquire a startup e amplia a integração do K6 à observabilidade, além dos testes de carga, consolidando o nome Grafana K6.

Made with GAMMA

O que torna o K6 útil no dia a dia

ARQUITETURA & RECURSOS

1 Executável leve em Go

BAIXO OVERHEAD

Consumo mínimo de CPU e memória na máquina geradora, viabilizando milhares de usuários virtuais em infraestrutura enxuta.

2 JavaScript & Protocolos

HTTP · WS · GRPC

Curva de aprendizado baixa com suporte nativo a HTTP/HTTPS, WebSockets e gRPC para APIs modernas e microsserviços.

3 Thresholds & Grafana

CI/CD & DASHBOARDS

Critérios objetivos que reprovam builds no pipeline e envio nativo de telemetria em tempo real para o ecossistema Grafana.

SCRIPTS COMO CÓDIGO FONTE · SEM INTERFACES PESADAS OU XML

Made with GAMMA

Antes de ir para K6 na prática, vamos entender os pilares do teste de performance

QUATRO PILARES

1 Latência

EXPERIÊNCIA

Quanto tempo uma operação leva do disparo até a resposta final recebida pelo cliente.

2 Throughput

VOLUME

Quantidade de trabalho processada por unidade de tempo. Uma métrica comum para isso é a quantidade de requisições por segundo.

3 Taxa de Erros

QUALIDADE

Proporção de falhas (status 4xx/5xx, timeouts e quebras de contrato) sob pressão contínua.

4 Recursos

CAUSA RAIZ

Uso de CPU, memória, pool de conexões, filas e saturação de rede para isolar o gargalo real.

RESPOSTA RÁPIDA NÃO TEM VALOR SE A TAXA DE ERROS ESTIVER AL

Made with GAMMA

Cada tipo de teste responde a uma pergunta diferente

Tipo	Objetivo	Como a carga se comporta	Pergunta que responde
Smoke	Verificar se o cenário mínimo funciona	Carga mínima, só para validar o básico	O cenário mínimo funciona?
Load	Medir o desempenho sob a carga esperada	Carga no nível esperado de uso	O sistema sustenta a carga esperada?
Stress	Observar o comportamento acima do esperado	Carga acima do que é esperado	O que acontece quando a carga ultrapassa o esperado?
Spike	Avaliar a reação a picos repentinos	Aumento abrupto de carga	Como reage a um aumento abrupto?
Soak	Detectar degradação ao longo do tempo	Carga constante por longo período	Mantém o desempenho por horas?
Breakpoint	Encontrar o limite de capacidade	Carga aumentada até os critérios de desempenho deixarem de ser atendidos	Em que ponto viola os critérios de desempenho?

Made with GAMMA

Revisão rápida: o que ficou da teoria?

Antes de partir para a prática, teste seu entendimento.

01

Qual era o gap que o K6 surgiu para resolver entre as ferramentas de teste existentes?

02

Quais são os quatro pilares principais que observamos em um teste de performance?

03

Em que ano o K6 surgiu?

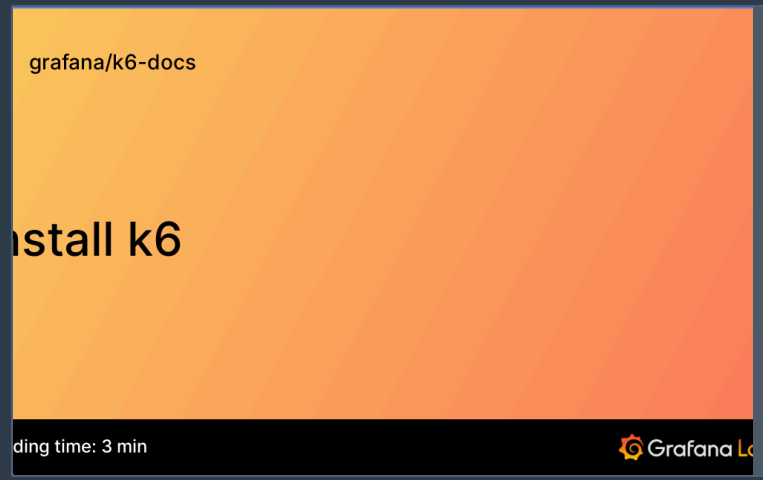
04

Como podemos definir teste de performance?

Made with GAMMA



Instalar o K6 é simples em qualquer ambiente



Grafana Labs

Install k6 | Grafana k6 documentation

k6 has packages for Linux, Mac, and Windows. As alternatives, you can also using a Docker container or a...

Windows

Chocolatey, Winget ou instalador MSI

Linux

APT para Debian/Ubuntu ou DNF para Fedora

macOS

Homebrew

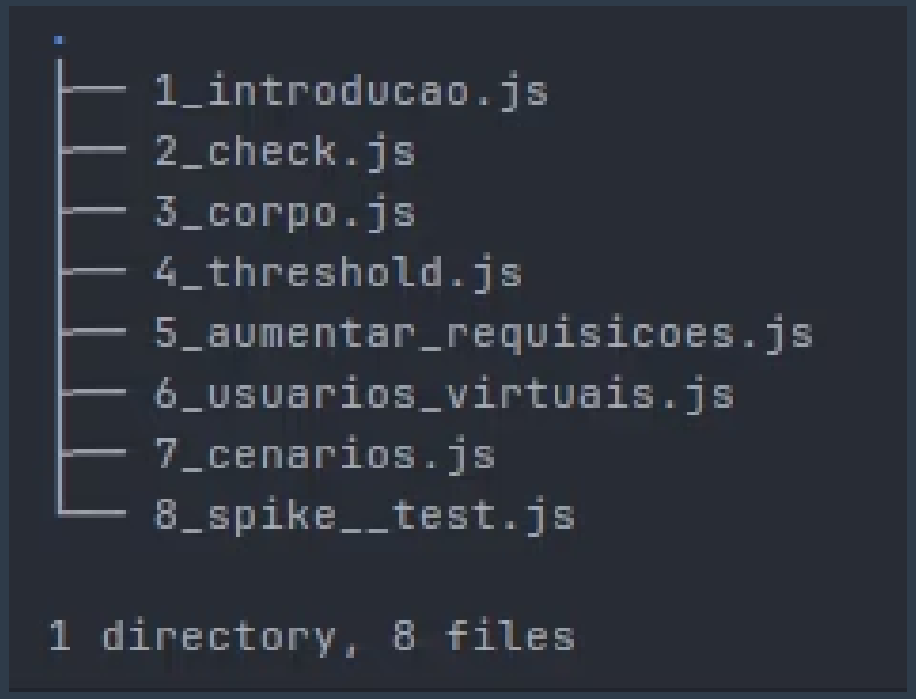
Alternativa multiplataforma

Docker com grafana/k6

Made with GAMMA

Arquitetura dos testes: uma evolução em scripts

Cada arquivo adiciona uma nova capacidade ao teste anterior.



1	1_introducao.js	primeira requisição e status
2	2_check.js	validação do status HTTP
3	3_corpo.js	validação do conteúdo
4	4_threshold.js	critérios de aprovação
5	5_aumentar_requisicoes.js	mais iterações
6	6_usuarios_virtuais.js	concorrência com VUs
7	7_cenarios.js	estratégias de carga
8	8_spike_test.js	teste de pico

Do request básico ao comportamento sob carga

Made with GAMMA

Primeiro script: faça uma requisição e observe o status

1. Verifique a instalação no terminal com `k6 version`.
2. Com uma API disponível para testar, crie um arquivo `.js`.
3. Execute o script para fazer uma requisição HTTP e registrar o status da resposta.

`import http from 'k6/http'` importa o módulo HTTP do K6, que permite usar métodos como `http.get()`, `http.post()`, `http.put()` e `http.del()`.

Em JavaScript, esse primeiro script apenas chama um endpoint do servidor e registra o status HTTP retornado.

`primeiro_teste.js`

```
import http from 'k6/http';

export default function () {
  const resposta =
http.get('http://localhost:8080/server');
  console.log(`Status HTTP: ${resposta.status}`);
}
```

`Execute no terminal:`

```
k6 run 1_introducao.js
```

Made with GAMMA

Três áreas, três perguntas diferentes

HTTP

Requisições ao servidor

`http_req_duration`: tempo desde o envio da requisição até o recebimento do corpo da resposta.

`http_req_failed`: percentual de requisições que falharam.

`http_reqs`: quantidade de requisições realizadas.

EXECUTION

Execução do script

`iteration_duration`: tempo total de uma iteração da função default, incluindo chamada, recebimento, processamento e finalização.

`iterations`: quantidade de iterações executadas.

Diferença do HTTP: uma iteração pode conter várias requisições.

NETWORK

Volume de dados

`data_sent`: volume enviado ao servidor, com corpos e cabeçalhos.

`data_received`: volume recebido, com corpos e cabeçalhos.

Trafega a cada chamada HTTP e ajuda a entender o consumo de banda.

HTTP MEDE A CHAMADA · EXECUTION MEDE O SCRIPT · NETWORK MEDE O VOLUME TRAFEGADO

Made with GAMMA

Métricas de tempo: média, mediana e percentis

Métrica	O que representa	Exemplo de leitura
avg	Média de todos os tempos: soma dividida pelo número de requisições.	avg de 200 ms pode esconder requisições bem mais lentas.
min	Menor tempo registrado, a resposta mais rápida do teste.	min de 80 ms: a melhor resposta foi em 80 ms.
med	Mediana: metade das requisições ficou abaixo dela e metade acima.	med de 150 ms: metade dos usuários esperou até 150 ms.
max	Maior tempo registrado, a pior espera do teste.	max de 3,2 s: alguém esperou 3,2 s por uma resposta.
p(90)	90% das requisições ficaram naquele tempo ou abaixo.	p(90) de 400 ms: 10% das requisições levaram mais de 400 ms.
p(95)	Mesmo raciocínio para 95% das requisições.	p(95) de 900 ms: 5% das requisições levaram mais de 900 ms.

PERCENTIS MOSTRAM A CAUDA: UMA MÉDIA BOA PODE ESCONDER 5% DOS USUÁRIOS ESPERANDO MUITO MAIS

Made with GAMMA

EVOLUÇÃO DO SCRIPT

Melhorando o script: a resposta precisa atender a uma condição

O primeiro script apenas fazia uma requisição ao servidor e imprimia o status, mas não verificava se a resposta atendia à expectativa.

1. Importar check de k6
2. Fazer a requisição HTTP
3. Verificar a condição “status HTTP é 200”
4. Considerar falha quando a resposta for diferente de 200

Antes: o script observava o resultado.
Agora: o script valida uma condição esperada.

`check()` registra a condição como aprovada ou falha; o critério formal de aprovação do teste será definido depois, com thresholds.

```
import http from 'k6/http';
import { check } from 'k6';

export default function () {
  const resposta =
http.get('http://localhost:8080/server');
  check(resposta, {
    'status HTTP é 200': (r) => r.status === 200,
  });
}
```

COMANDO · $ K6 RUN PRIMEIRO_TESTE.JS

Made with GAMMA

VALIDAÇÃO DO CONTEÚDO

Validando o conteúdo: a resposta também precisa fazer sentido

Com o mesmo check(), podemos adicionar uma segunda condição para verificar o corpo da resposta, não apenas o status HTTP.

Comunicação HTTP

A requisição respondeu com status 200?

Comportamento funcional

O corpo contém a mensagem esperada?

Agora: o teste deixa de observar apenas se houve comunicação e começa a verificar se a aplicação respondeu corretamente do ponto de vista funcional.

```
import http from 'k6/http';
import { check } from 'k6';

export default function () {
  const resposta =
http.get('http://localhost:8080/server');

  check(resposta, {
    'status HTTP é 200': (r) => r.status ===
200,
    'corpo possui mensagem esperada': (r)
=> r.body.includes('server is running'),
  });
}
```

Made with GAMMA

THRESHOLDS

Thresholds: transformando métricas em critérios de aprovação

Podemos definir uma constante **options** e, dentro dela, os critérios de aceitação do teste.

No exemplo, **http_req_duration** com **p(95)<500** significa que 95% das requisições devem ter tempo de resposta menor ou igual a 500 milissegundos. A mesma lógica pode ser aplicada a outros indicadores que vimos anteriormente.

- **check():** verifica uma condição em uma resposta e registra se ela foi satisfeita.
- **threshold:** define o critério de aceitação do teste e indica se o resultado global passou ou falhou.

Quando o threshold não é atendido, o resultado do teste é identificado como falha, e o K6 mostra qual critério foi violado.

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  thresholds: {
    http_req_duration: ['p(95)<500'+],
  },
};

export default function () {
  const resposta =
http.get('http://localhost:8080/server');

  check(resposta, {
    'status HTTP é 200': (r) => r.status === 200,
    'corpo possui mensagem esperada': (r) =>
r.body.includes('server is running'),
  });
}
```

Made with GAMMA

ITERAÇÕES

Aumentando a quantidade de requisições

- Antes, o script executava uma única iteração e fazia uma requisição ao servidor.
- Ao adicionar **iterations** dentro de **options**, definimos quantas vezes a função default será executada.
- Como este script faz uma requisição por iteração, **iterations: 100** resulta em até 100 requisições ao endpoint.
- Depois da execução, observamos o efeito em **http_reqs**, **iterations**, **http_req_duration** e **http_req_failed**.

Configuração	Execuções da função default	Requisições ao servidor
Sem iterations (1 iteração)	1	1
iterations: 100	100	até 100

```
export const options = {
  iterations: 100,
  thresholds: {
    http_req_duration: ['p(95)<500'],
  },
};

export default function () {
  const resposta =
http.get('http://localhost:8080/server');

  check(resposta, {
    'status HTTP é 200': (r) => r.status === 200,
    'corpo possui mensagem esperada': (r) =>
r.body.includes('server is running'),
  });
}
```

Made with GAMMA

VUS

Usando usuários virtuais para simular concorrência

A propriedade **vus** indica quantos usuários virtuais participarão do teste e permite simular diferentes usuários acessando o mesmo endpoint ao mesmo tempo.

ANTES

1 VU

1 usuário virtual executa as 100 iterações em sequência.

DEPOIS

10 VUs

10 usuários virtuais compartilham as 100 iterações e podem executar suas operações concorrentemente.

- Nesse modelo de iterações compartilhadas, o K6 distribui o trabalho entre os VUs.
- A distribuição não precisa ser uniforme: um VU pode executar mais iterações do que outro.
- VUs representam concorrência, não necessariamente 10 vezes mais iterações.

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 10,
  iterations: 100,
  thresholds: {
    http_req_duration: ['p(95)<500'],
  },
};

export default function () {
  const resposta =
http.get('http://localhost:8080/server');

  check(resposta, {
    'status HTTP é 200': (r) => r.status === 200,
    'corpo possui mensagem esperada': (r) =>
r.body.includes('server is running'),
  });
}
```

Made with GAMMA

VUS

Scenarios: controlando como a carga é gerada

Antes, **vus** e **iterations** definiam uma configuração simples e fixa de usuários virtuais e iterações. Com **scenarios**, podemos definir diferentes estratégias de execução e controlar a geração de carga com mais flexibilidade.

O executor **ramping-vus** controla a evolução dos VUs por etapas (**stages**). Cenários diferentes podem usar configurações independentes no mesmo teste.

- Aumentar ou diminuir a quantidade de usuários virtuais progressivamente
- Simular picos repentinos de acessos
- Manter uma carga constante durante várias horas
- Executar diferentes cenários no mesmo teste, com configurações independentes
- Controlar a duração e a intensidade da carga

```
export const options = {
  scenarios: {
    stress_test: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '10s', target: 5 },
        { duration: '5s', target: 10 },
        { duration: '10s', target: 100 },
        { duration: '10s', target: 150 },
      ],
    },
  },
};
```

Made with GAMMA

Executors: quem recebe o trabalho?

shared-iterations

10 VUs compartilham 1.000 iterações. O cenário termina quando as 1.000 acabam, e a distribuição não precisa ser uniforme.

```
executor: 'shared-iterations',
vus: 10,
iterations: 1000
```

Use quando existe uma quantidade total de trabalho a concluir.

per-vu-iterations

Cada VU executa sua própria meta. Com 10 VUs e 100 iterações por VU, o total esperado é 1.000.

```
executor: 'per-vu-iterations',
vus: 10,
iterations: 100
```

Use quando cada VU precisa executar uma quantidade específica.

Diferença: trabalho total compartilhado versus meta individual por VU.

Made with GAMMA

VUs constantes ou em rampa?

constant-vus

Mantém uma quantidade fixa de VUs, por exemplo 10 VUs durante 5 minutos. Cada VU repete a função default; a taxa de requisições depende da velocidade das iterações.

```
executor: 'constant-vus',
vus: 10,
duration: '5m'
```

ramping-vus

Aumenta ou diminui a quantidade de VUs ao longo do tempo. É útil para estudar o comportamento conforme a carga cresce ou diminui, em testes de carga e estresse.

```
executor: 'ramping-vus',
startVUs: 0,
stages: [
  { duration: '30s', target: 10 },
  { duration: '1m', target: 10 },
  { duration: '30s', target: 50 },
  { duration: '30s', target: 0 },
]
```

stages define etapas com **duration** e **target**. O ramping-vus controla VUs, não uma quantidade exata de re
segundo.

Made with GAMMA

Arrival-rate: controlando a taxa de chegada

constant-arrival-rate

Inicia uma taxa definida de iterações, por exemplo 20 por segundo durante 1 minuto. **preAllocatedVUs** define os VUs iniciais e **maxVUs** o limite. Se as respostas ficarem lentas, mais VUs podem ser necessários; sem VUs suficientes, iterações podem não iniciar como planejado.

```
executor: 'constant-arrival-rate',
rate: 20,
timeUnit: '1s',
duration: '1m',
preAllocatedVUs: 10,
maxVUs: 50
```

ramping-arrival-rate

Começa em uma taxa e aumenta por stages, por exemplo de 10 para 50 iterações por segundo. Difere do ramping-vus porque aumenta a taxa de início das iterações, não a quantidade de usuários.

```
executor: 'ramping-arrival-rate',
startRate: 10,
timeUnit: '1s',
preAllocatedVUs: 10,
maxVUs: 100,
stages: [
 { duration: '1m', target: 10 },
 { duration: '1m', target: 50 },
 { duration: '1m', target: 50 },
]
```

Iterações por segundo não equivalem necessariamente a requisições HTTP por segundo: cada iteração pode fazer uma ou várias requisições.

Made with GAMMA

MATRIZ

Escolha o executor pela pergunta do teste

Executor	O que controla	Quando usar
shared-iterations	Quantidade total de trabalho	Execução pontual com volume definido
per-vu-iterations	Meta individual por usuário	Comparar comportamento por VU
constant-vus	Quantidade fixa de usuários	Observar carga constante
ramping-vus	Quantidade de usuários ao longo do tempo	Load e stress tests
constant-arrival-rate	Taxa fixa de início de iterações	Controlar a chegada de trabalho
ramping-arrival-rate	Taxa de chegada crescente ou decrescente	Estudar capacidade sob chegada variável

Regra: load, stress, spike e soak não são apenas nomes. São estratégias de carga construídas combinando executor, duração, stages, VUs e taxa de chegada. As configurações são a base para montá-las.

Made with GAMMA

Testes de performance automatizados

- Executar testes de performance de forma repetível
- Transformar métricas e thresholds em uma decisão automática de passou ou falhou
- Integrar a execução ao CI/CD para acompanhar a performance continuamente

1
Script

2
Execução

3
Métricas e thresholds

4
Decisão

5
CI/CD

Made with GAMMA

Obrigado pela atenção

Mapa mental usado para planejar a aula: https://whimsical.com/tagashi/k6-UpoGdhhKvUij5Gn6cFhk6d

Repositório usado como exemplo de teste de performance e automação: https://github.com/taagashi/Chamados-IFRN

Made with GAMMA